Cloud Computing Rair Documentation

Group number: 6

Aymerick Tilly (Team leader)
Student ID: 24244239
Email: vkx0580@autuni.ac.nz

Ronan Gondipon
Student ID:23227991
Email: grr9624@autuni.ac.nz

Hafizur Rahman
Student ID: 24236822
Email: srj7826@autuni.ac.nz

Isaac Kuku
Student ID: 19085865
Email: vdy0971@autuni.ac.nz

June 6, 2025

Contents

1 Rair and GitHub URI

Rair URI	GitHub URI
https://rair-shop.xyz	https://github.com/AymerickTilly/Cloud-Computing

Table 1: Rair and GitHub URI

2 Credential Information Admin and User

To simulate a real-world application, Admin and User groups were created in the Cognito user pool. To enable testing of admin-specific commands and UI, the admin account is provided in Table 2. Similarly, a user account is provided in Table 3 to streamline the registration and login process.

Username	Password
rairAdmin1@gmail.com	PasswordAdmins2025#

Table 2: Admin Account Credential

Username	Password
rairUser1@gmail.com	PasswordUsers2025#

Table 3: User Account Credential

3 Introduction

Rair Clothing is a web platform connecting fashion enthusiasts in New Zealand, allowing us to showcase projects, promotions, and updates. Customers can personalise profiles, browse the store, add items to their cart, and view order history. For creators, the platform offers better stock visibility, reducing overstock and financial losses. Ultimately, the physical store will focus on fitting, while customers can easily purchase merchandise online.

4 Project Planning

Scrum is the framework utilised for the design, development, and testing phases of the project. By using standardised procedures like daily meetings, sprint planning, reviews, and retrospectives, Scrum promotes a culture of continuous development and adaptability to shifting requirements. Trello was utilised to track and manage the progress of the project with weekly updates

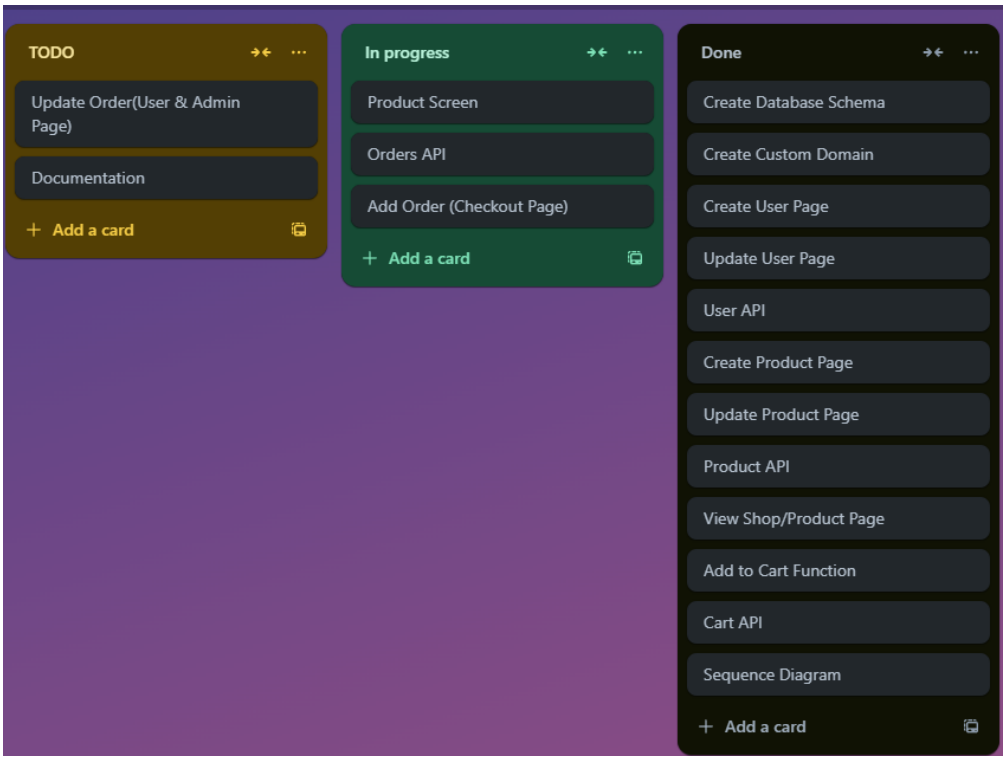

Figure 1: Figure 1

5 Project Design

5.1 User Experience and Screen Mock-ups

Screen mock-ups visually represent the RAIR user interface, offering stakeholders a clear preview of the platform's layout, features, and user flows. These early design concepts support collaborative feedback and iterative refinement, ensuring alignment between technical implementation and user expectations. While mock-ups may differ from the final deployed product, they serve as a critical step in the ideation and development process.

The user journey on RAIR begins with account registration, where individuals provide their name, email address, delivery address, and password. Upon successful sign-up, users are grouped under the "Customer" category by default. Once registered, users can log in to access personalised features such as browsing products, managing their cart, checking out, and viewing order history.

Sign-up Page

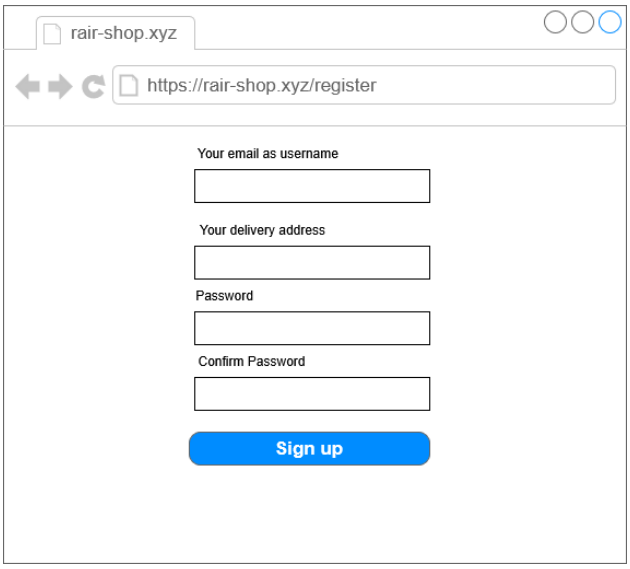

Figure 2: Sign-up page

Start with the sign-up screen. A new user enters their full name, email address, delivery address, and password. All fields are mandatory, and the user must confirm their password to ensure it's entered correctly. After clicking 'Sign Up, their information is securely registered. They are then redirected to the login screen. This process ensures that every purchase is verified and uniquely identified.

Login Page

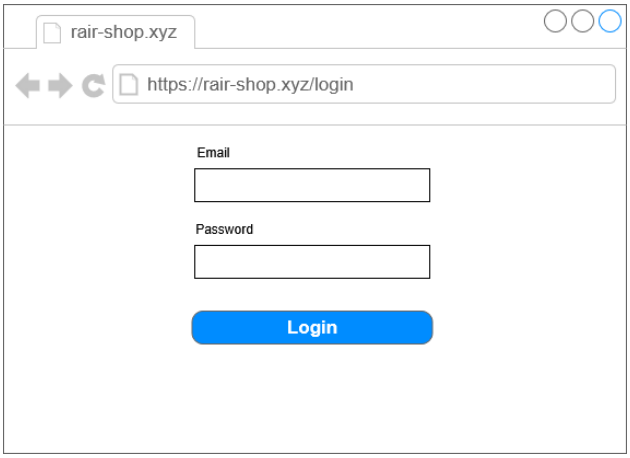

Figure 3: Login Page

After creating an account, the user can log in using the same email and password they chose during sign-up.

Home Page

Figure 4: Home Page

Shop Page

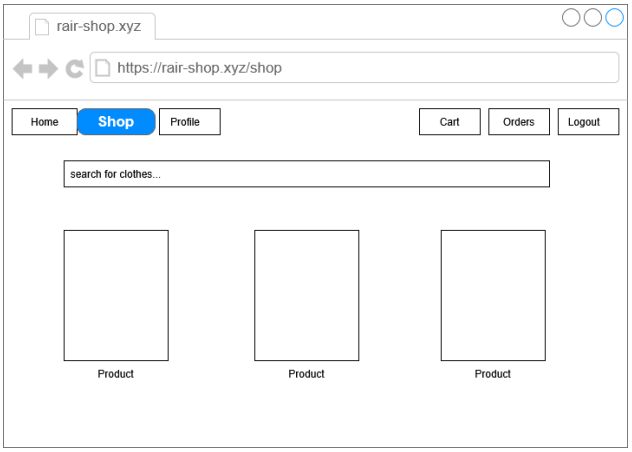

Figure 5: Shop Page

Once the user logs in, they're taken to the home page and then the shop page. This is where all the products are shown. Featured items appear in a carousel at the top, and below that is a list of everything available. Each product is shown with a picture, name, and price. The user can scroll to browse or use the search bar to find something specific.

Profile Page

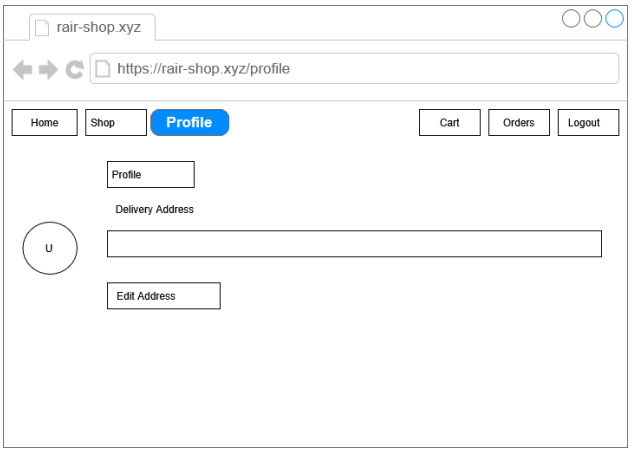

Figure 6: Profile Page

This is the profile page where users can update their delivery address if needed.

Cart Page

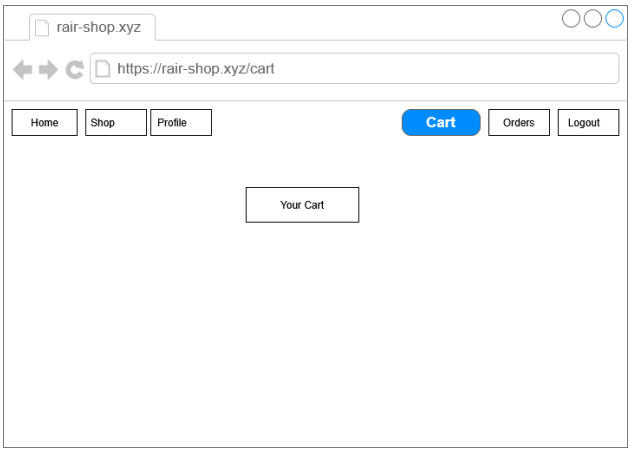

Figure 7: Cart Page

From the product detail view, the user selects a size, chooses the quantity, and clicks the 'Add to Cart' button. A brief confirmation message appears, letting them know the item has been successfully added.

Order Page

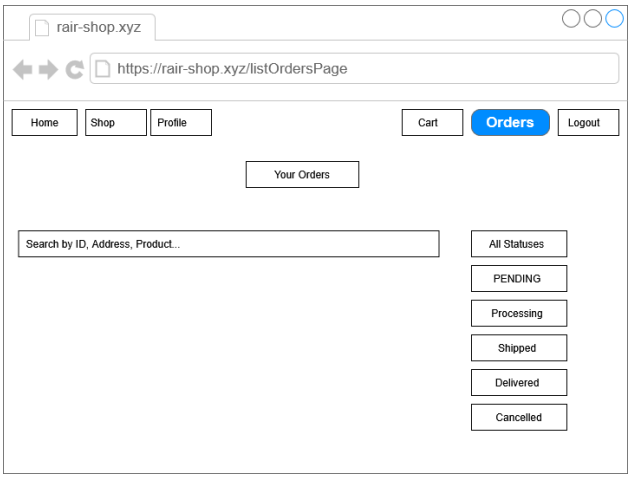

Figure 8: Order Page

By clicking the cart icon in the top navigation, the user accesses their cart. Here, they can see each item they've added, along with the size, quantity, and price. They can update quantities or remove items as needed. The total cost is updated automatically. When ready, they proceed by clicking the 'Checkout' button. If the user is in the customer category, they will only be able to see their orders, while the administrators can view all orders made.

Checkout Page

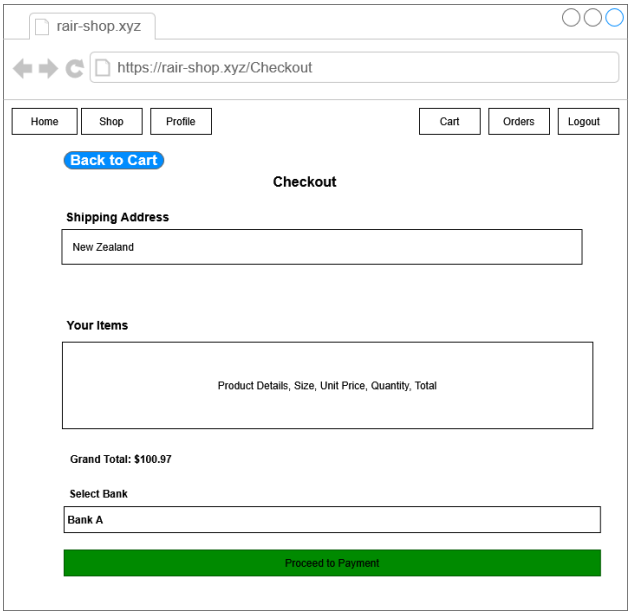

Figure 9: Checkout Page

The checkout page displays a summary of the order, including selected products, delivery address, and total amount. The user selects their preferred payment method, confirms everything, and clicks 'Place Order.' Then the user sees a confirmation message or page.

Admin Dashboard Page

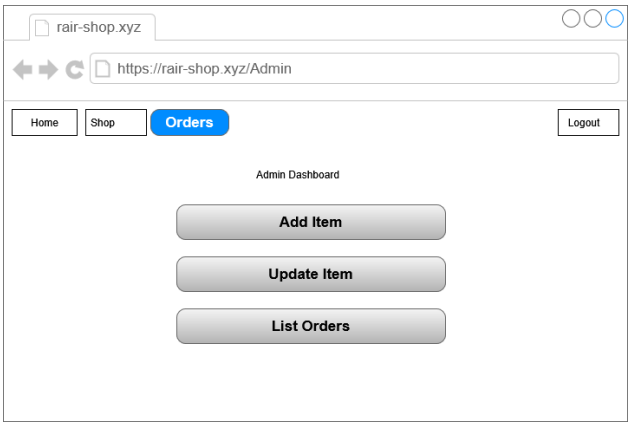

Figure 10: Admin Page

On the Admin dashboard, they can add new products, update or delete existing ones, and view all customer orders. The dashboard has buttons to go to each of these actions.

Update Product Modals Page

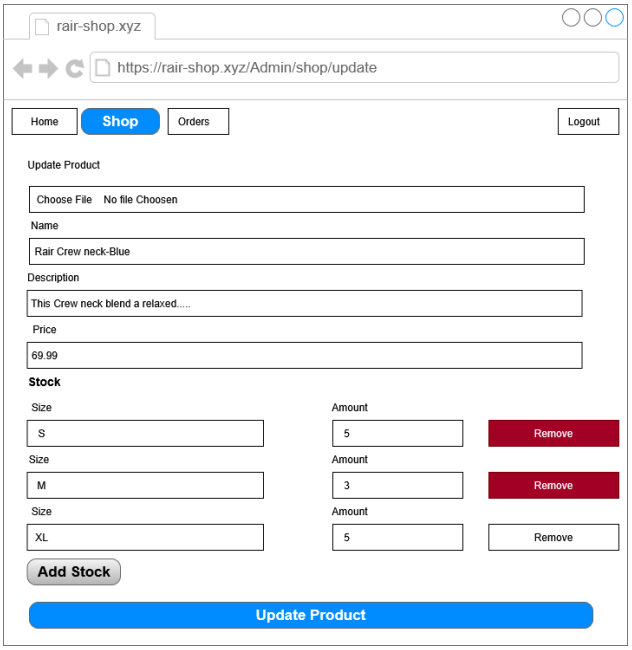

Figure 11: Update Product Modals Page

The admin sees a catalogue of products similar to the shop page. When they click 'Update' on a product, a modal (pop-up) appears. Inside it, they can edit the product's name, description, stock quantity, and image. If needed, they can also delete the product from the system.

6 Cloud Implementation

6.1 Cloud Services Architecture

Amazon Web Services (AWS) was selected as the cloud platform for the Rair shop project. AWS offer reliable, cost-efficient infrastructure and a wide range of services. The Rair project aims to maintain high availability, enable scalability and data privacy.

As a user authentication and authorisation service, AWS Cognito was employed, resulting in an easier identity management service. Combined with Cognito, AWS Amplify in-built functions — Sign In, Sign Up, Sign Out, Confirm Sign Up, and Forgot Password—were used to manage the user authentication and authorisation process with Cognito. To manage the authorisation functionalities, two Cognito groups were created, namely 'Customer' and 'Admin'. When a user registers, he is automatically assigned to the Customer group. Moreover, Multi-Factor Authentication (MFA) can be employed to enhance user account security and recovery.

AWS Simple Storage Service (S3) provides a reliable platform for storing and retrieving objects such as files and websites. Two S3 buckets were used to host the Rair web page in the first place, statically and to store Rair product images. High durability and availability are ensured because of its distributed architecture spanning multiple geographic regions, making it suitable for supporting an overseas customer base, ensuring access to the Rair Clothing platform for both customers and creators.

AWS API Gateway service was leveraged for the Rair project to define RESTful interfaces that handle CRUD (create, read, update, delete) operations for products, orders, carts, and user accounts. It simplifies secure communication between the front-end and backend by managing authentication and authorisation.

AWS DynamoDB is a NoSQL database delivering consistent and fast performance, making it ideal for projects needing low-latency access to data. In the Rair project, it was used to store and manage product details, user details, cart contents, and order histories efficiently.

AWS Lambda is a serverless service that executes code in response to events, usually triggered from an API Gateway request, automatically handling scaling and infrastructure. It was used to provide backend functions, such as image uploading, requesting, order updates, and inventory management, using Node.js version 22.

AWS Route 53 is a DNS service that maps domain names to IP addresses. In the Rair project, it was used to manage the custom domain rair-shop.xyz domain and route the traffic efficiently.

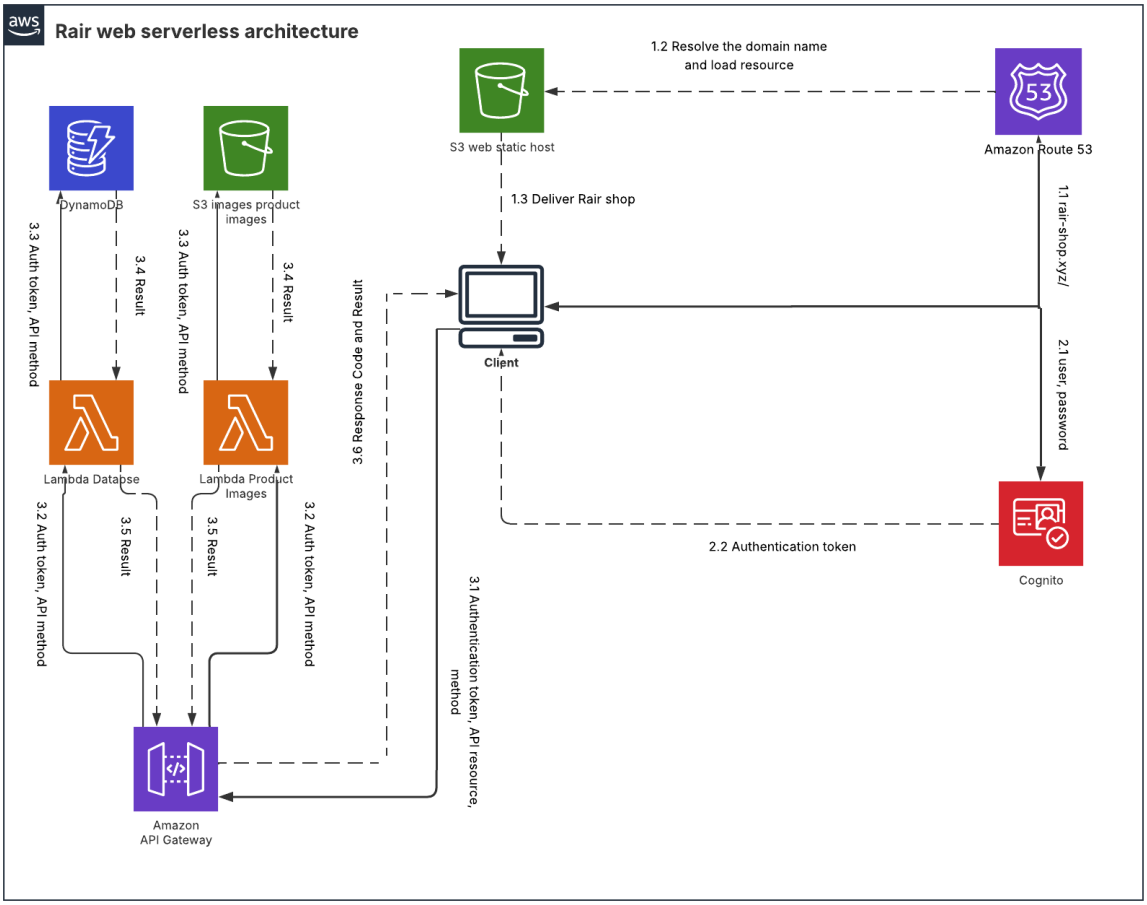

Figure 12: AWS Cloud Integration: This serverless architecture outlines the flow of cloud services used by the RAIR website.

6.2 Technology Stack

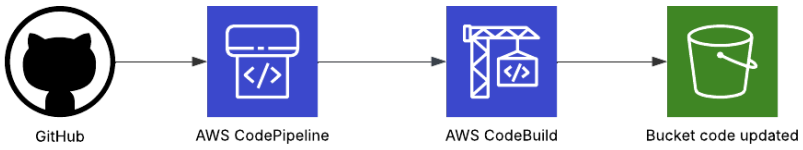

Figure 13: Continuous Integration and Development Schema

The Lambda functions were developed using Node.js version 22. To enhance the scalability, each RESTful interface was separated into different files, making it easier to handle for future improvements. React and TypeScript were used to build a dynamic and interactive front-end interface. React Bootstrap was used to facilitate the styling.

GitHub was used for source control, enabling version tracking and smooth collaboration among team members. AWS CodePipeline was integrated with the GitHub branch to automate the build and deployment process. It triggers the pipeline whenever changes are pushed, and updates to the S3 bucket are made only if the build process through AWS CodeBuild is successful, ensuring reliable and consistent delivery of the application.

As illustrated in Figure 14, the latest update to the Rair application successfully passed through the entire CI/CD pipeline. This demonstrates that the automated workflow, starting with source code changes committed to GitHub, followed by build validation using CodeBuild, and concluding in the S3 bucket, was executed correctly.

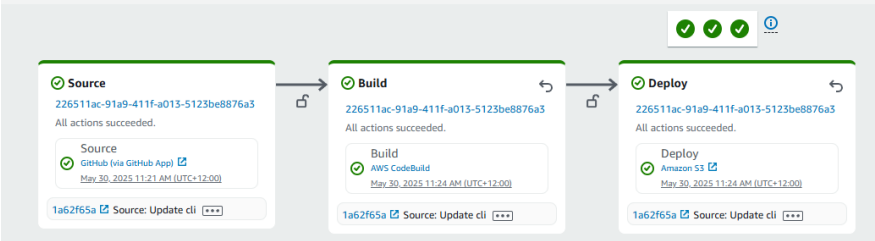

Figure 14: Continuous Integration and Development

6.3 Interactions and Sequence Diagrams

To access the Rair application, a user first enters their credentials, which Cognito verifies. Upon successful authentication (and authorisation), Cognito issues the ID, Access, and Refresh tokens. The front-end then calls a REST endpoint by including the Access token in the request. API Gateway validates the token against Cognito and, if the token is verified, forwards the call to the appropriate Lambda function. Lambda interacts with DynamoDB or S3, depending on the requested resource to retrieve or update data. Finally, the processed data is returned through the API Gateway and delivered to the user's browser.

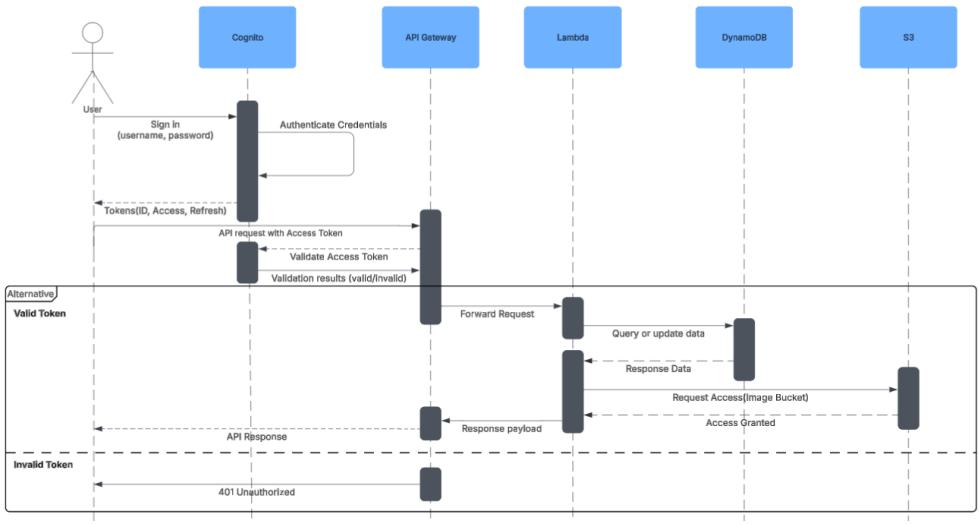

Figure 15: Sequence Diagram

6.4 DynamoDB Schema Design

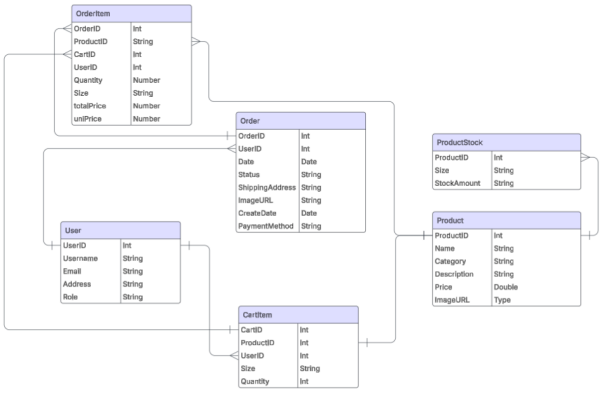

Figure 16: Entity Relationship Diagram

The schemas for the Cart, Order, User, and Product tables define the structure and organisation of data within the Rair system, enabling efficient storage, retrieval, and management of e-commerce transactions and user activities.

6.4.1 User table

The User Table schema centralises user identity and delivery information. The userId serves as a globally unique identifier for linking user-related operations across all tables. The address field stores the user's primary delivery location, and the username field represents the display name associated with the account, aiding in personalised communication and UI rendering.

```
{
    "userId": "String",
    "address": "String",
    "username": "String"
}
```

6.4.2 Cart table

The Cart Table schema outlines the attributes required to maintain a user's shopping cart. Each cart item is uniquely identified by the cartId field, which allows for the differentiation of identical products added multiple times. The userId field associates each cart entry with a specific user, facilitating personalised cart management. The schema also includes product-specific attributes such as imageUrl, name, price, and productId, which capture the visual and descriptive details of the item as well as its original reference in the Product Table. Furthermore, fields like quantity and size reflect user-selected configurations, enabling accurate cart summaries and order preparation.

```
{
    "userId": "String",
    "cartId": "String",
    "imageUrl": "String",
    "name": "String",
    "price": "Number",
    "productId": "String",
  "quantity": "Number",
  "size": "String"
}
```

6.4.3 Order table

The Order Table schema captures all relevant information for processing and tracking orders. Each order is uniquely identified by the orderId field, while the date records the timestamp of the order placement. The paymentMethod field indicates how the transaction was completed using which bank. A nested products array contains detailed objects for each item in the order. This includes the cartId and productId, and descriptive information such as name, image, quantity, size, unitPrice, and totalPrice. Additional fields such as shippingAddress and status support, while totalAmount reflects the total cost of the order. User-related fields like userId and username link the order to the customer for reference and display.

```
{
    "orderId": "String",
    "date": "String",
    "paymentMethod": "String",
    "products": [
    {
        "cartId": "String",
        "id": "String",
        "image": "String",
        "name": "String",
        "quantity": "Number",
        "size": "String",
        "totalPrice": "Number",
        "unitPrice": "Number"
    }
  ],
    "shippingAddress": "String",
    "status": "String",
    "totalAmount": "Number",
    "userId": "String",
    "username": "String"
}
```

6.4.4 Product table

Lastly, the Product Table schema defines the essential structure for managing the platform's catalogue. Each product is uniquely referenced by the productId, and categorised using the category field for browsing and filtering purposes. The description, imageUrl, and name fields provide the necessary textual and visual content for listing and displaying products. The price field holds the cost per unit, while the stock attribute is an array of objects which specify the availability of each size variant through the size and stockAmount subfields. This design supports dynamic inventory tracking and ensures size-specific stock management.

```
{
    "productId": "String",
    "category": "String",
    "description": "String",
    "imageUrl": "String",
    "name": "String",
    "price": "Number",
    "stock": [
        "size": "String",
        "stockAmount": "Number"
        ]
}
```

7 User Manual

Login Page

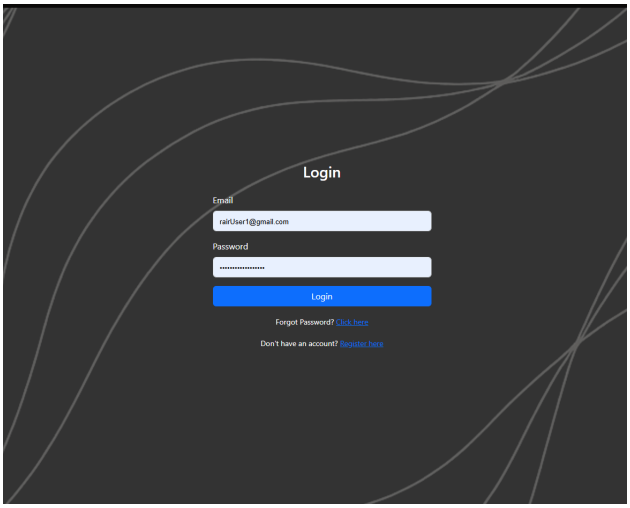

Figure 17: Login Page

The login screen allows the user to log in to their account simply by entering their registered email and password. If the user is new, they can click on register to create their account. After logging in, the user will be brought the the home page. Depending on the assigned user group, some buttons will not be visible to the user.

Registration page

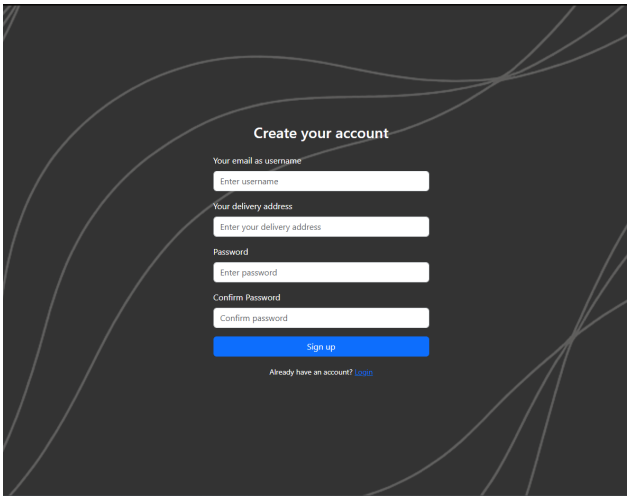

Figure 18: Registration Page

The registration page allows the user to register an account simply by entering their email, delivery address, password and confirm password. Once the required fields are filled, click on the Sign up button, and after processing, the screen will be brought to the login page.

Reset Password From Email

This page allows the user to request a password reset code from Cognito. If the provided email matches an existing user in the Cognito user pool, a reset code will be sent to that email, and the user will be redirected to the password reset confirmation page.

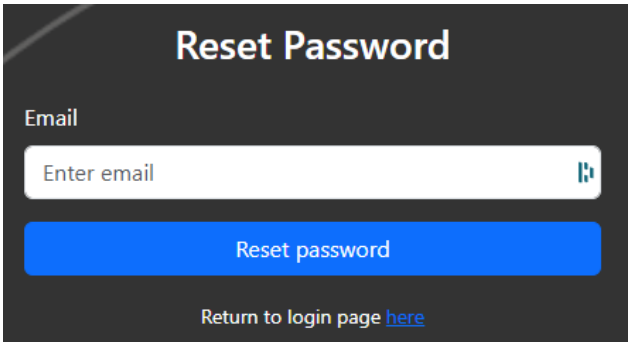

Figure 19: Reset password from Email

Reset Password

On this page, users must enter the confirmation code received via email (in Figure 21) along with their new password and confirm it. Upon successful password reset, the user is redirected to the login page, and their password is updated.

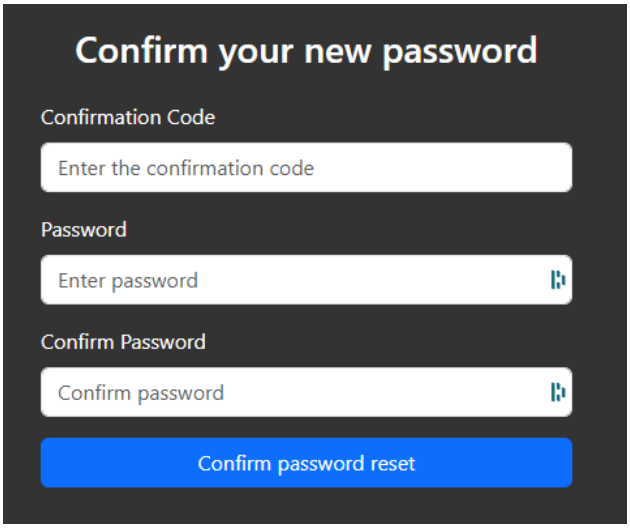

Figure 20: Confirm Password Reset

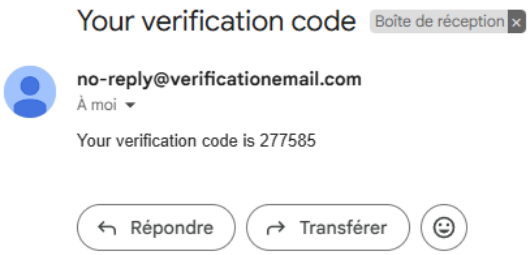

Figure 21: Email Reset Password Confirmation Code

Home Page

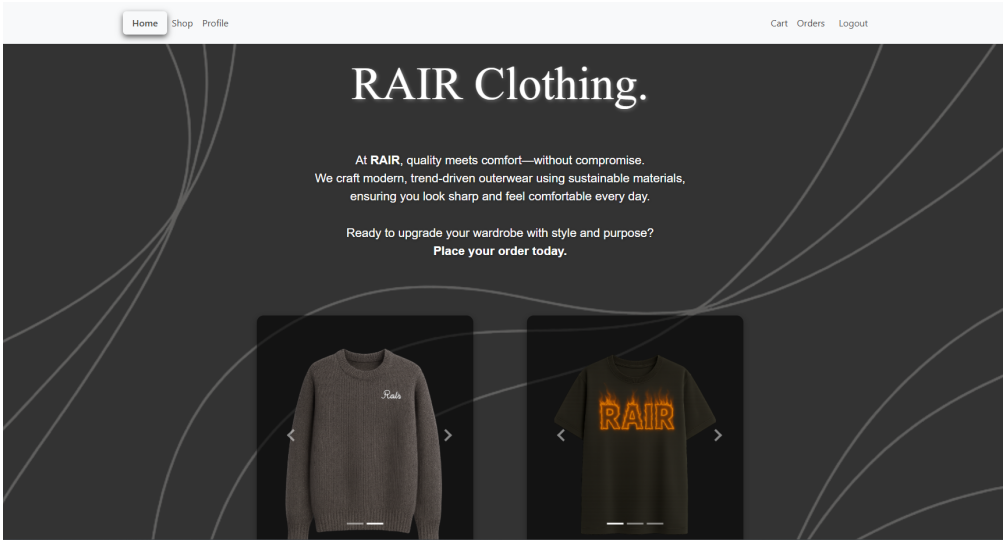

Figure 22: Home Page

This is the page that the user will be directed to upon logging in. There are several carousels that show the products; clicking on them will reveal more details about the product.

Shop page

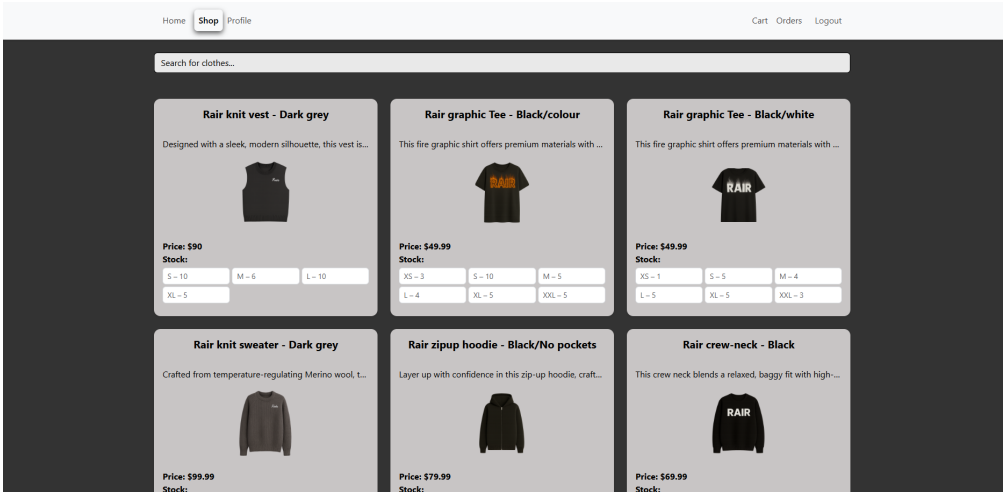

Figure 23: Figure 3

In this screen, the page will show the catalogues of various products. A search bar is provided for the user to search for something specific.

7.1 Shop functionality

Clicking on an item will show its details, which include the name, description, and size. The user can add the item to the cart after selecting the size and clicking the **"add to cart"** button.

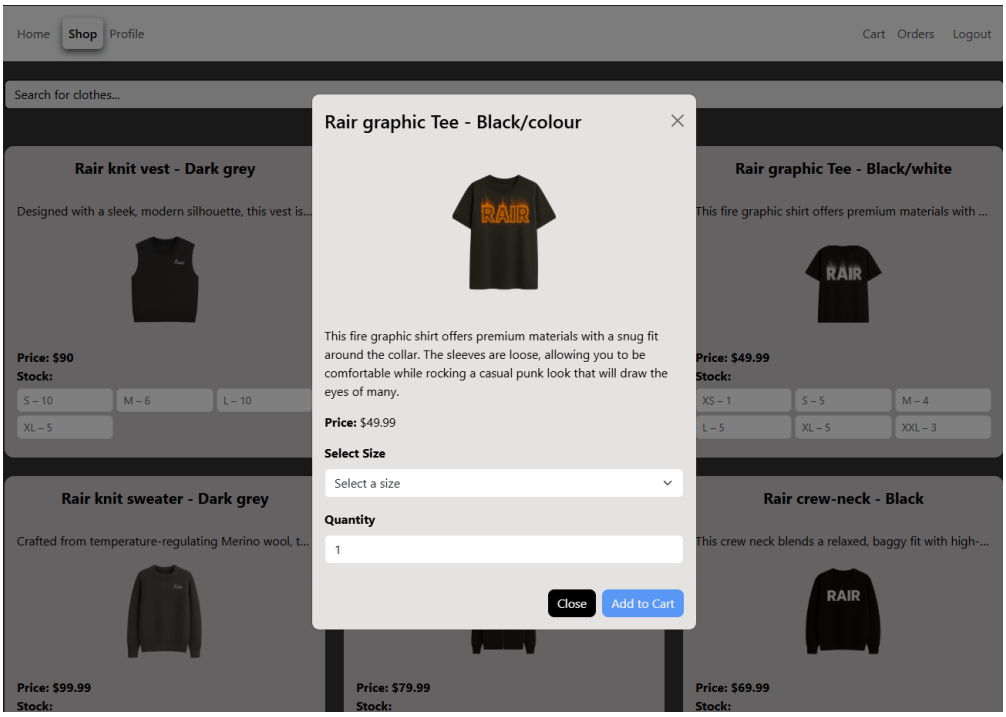

Figure 24: Product Modal figure 4

Profile page

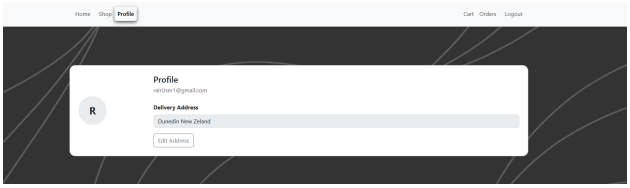

Figure 25: Profile Page

This page is the profile page, which will allow the user to change their delivery address.

Cart page

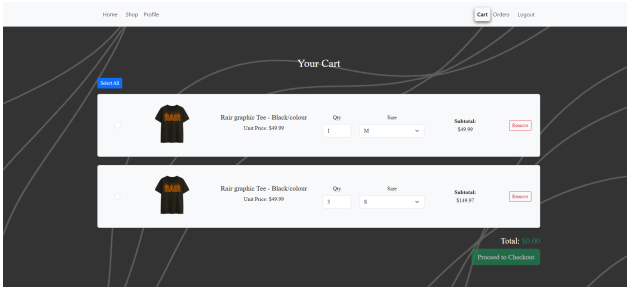

Figure 26: Cart Page

This page will show a list of all items added to the cart by the user. The user can tick the desired items to buy and adjust their quantity accordingly. Once the items are selected, the user can click the "Proceed to Payment" button, which directs them to the Payment Page. If the amount in the quantity exceeds the current existing stock, the system will not allow the user to proceed and will be prompted with a message telling the customer to current number of stock available.

Check Out Page

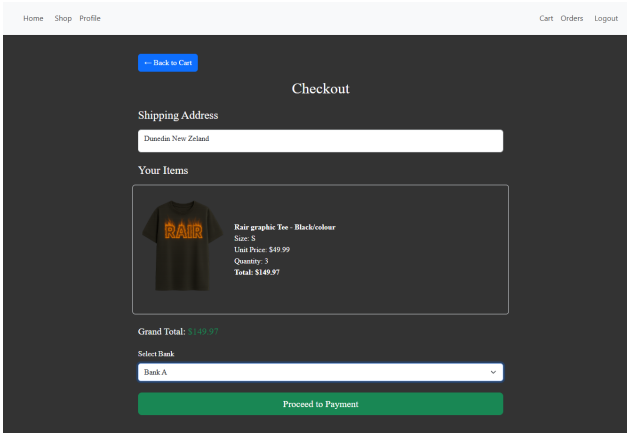

Figure 27: Checkout Page

This page will allow the user to view what they will be going to order, select payment options and confirm their payments.

Orders page

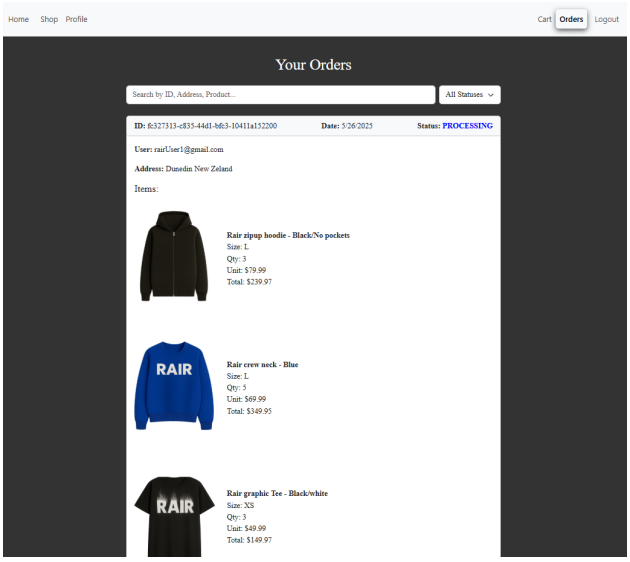

Figure 28: Orders Page

This page will show a list of orders created by the user. Details of each order include the delivery address, the items ordered, including their image, name, size, quantity and price. The total price will also be displayed alongside its status. If the status is still being processed, the user is able to click on cancel the order.

Admin Dashboard

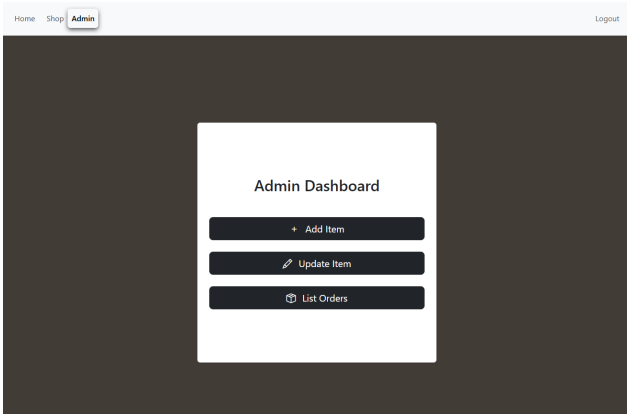

Figure 29: Admin Dashboard

The admin dashboard leads to 3 main functions, which are Add Item, which is used to add a new product, Update Item, which is used to update existing products, maintain stock, and List Orders, which is used to see existing orders and their statuses.

Add New Product Page

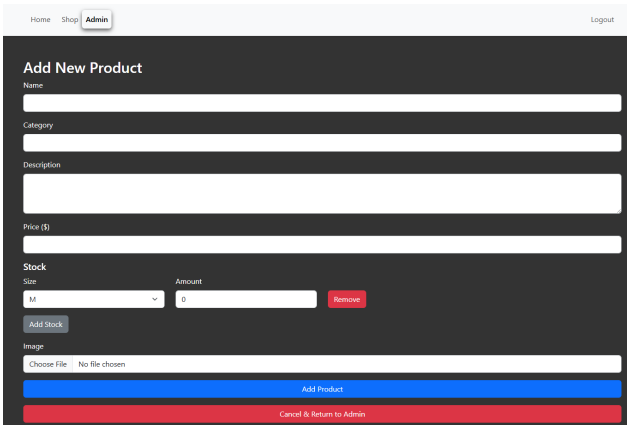

Figure 30: Add New Product Page

On this page, the staff member can upload the photo and details about the new product, such as its name, description, category, sizes available and quantity in stock for each size.

Update product page

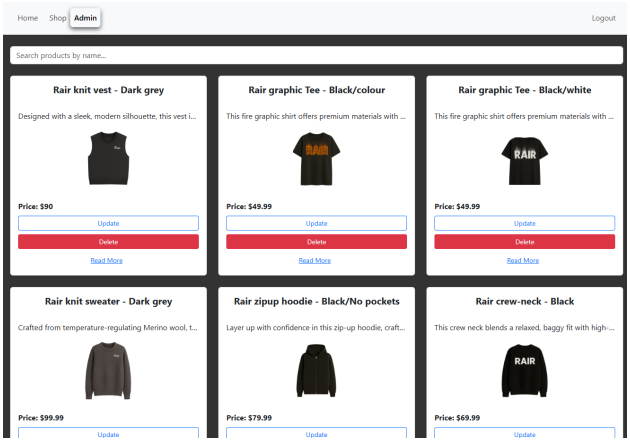

Figure 31: Update Product Page

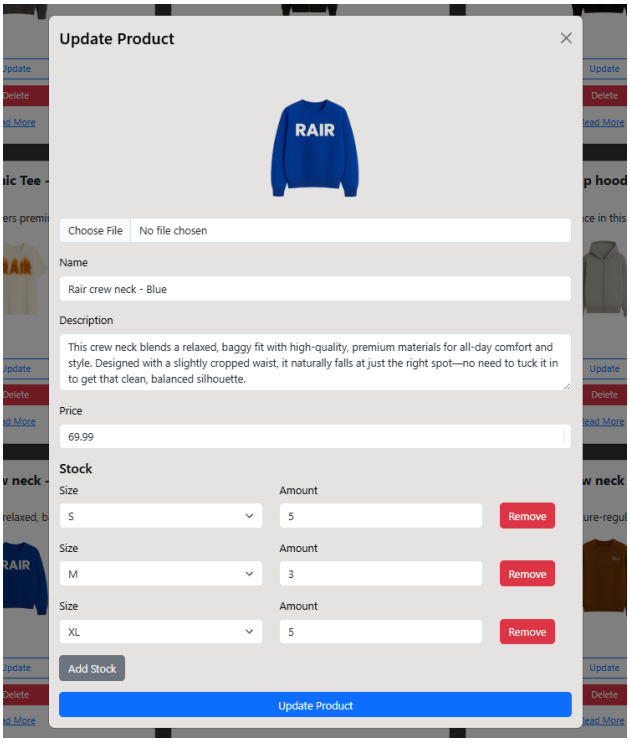

Figure 32: Update Product Modal

On this page, a catalogue of products similar to the shop page will appear. Clicking on the update button will reveal a modal that will allow the user to update the name, description, available stock quantity and image of the product. Admins can also delete the product if necessary.

View Orders Page.

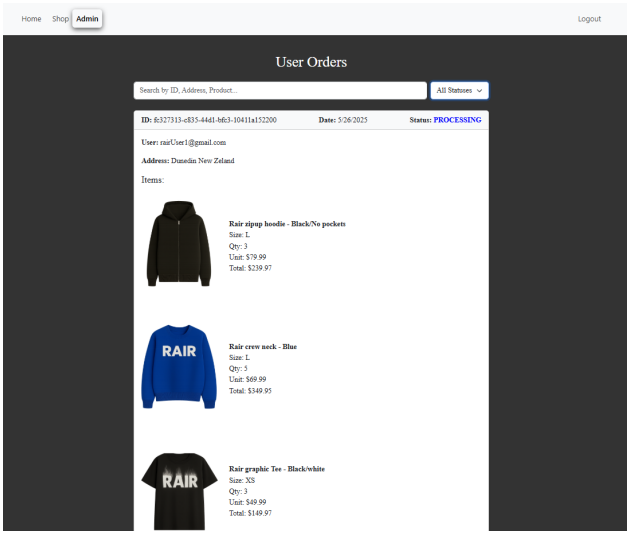

Figure 33: View Orders(Admin View)

Similarly to the user's view, this page allows the admin to view all existing orders and allows them to update the status of each order.

8 RESTful API List

The RESTful APIs utilise HTTPS to ensure secure communication, safeguarding the integrity and privacy of data transmitted across the network. By leveraging HTTPS encryption, the system blocks unauthorised access and interception of sensitive information, improving its overall security.

In the Rair project, the default API Gateway invoke URL https://yv9hvyex77.execute-api.ap-southeast-2.amazonaws.com was used instead of the api.rair-shop.xyz custom domain. This decision was driven by the fact that our API Gateway is hosted in the Sydney (ap-southeast-2) region, where AWS Certificate Manager cannot be used. Due to time constraints, migrating all APIs to the US East (N. Virginia) region and reconfiguring the entire authorisation framework and roles was not a doable option at this stage.

Endpoints	Method	Action
API URL/dev/user	POST	Create a user
API URL/dev/user?userId={userId}	GET	Get a user by Id
API URL/dev/users	GET	Get all users
API URL/dev/user	PUT	Update a user

Table 4: REST endpoints Users

Endpoints	Method	Action
API URL/dev/image	POST	Create an image
API URL/dev/image?imageUrl={imageUrl}	DELETE	Delete an image by matching Url

Table 5: REST endpoints Images

Endpoints	Method	Action
API URL/dev/product	POST	Create a product
API URL/dev/product?productId={productId}	GET	Get a product by Id
API URL/dev/products	GET	Get all products
API URL/dev/product	PUT	Update a product
API URL/dev/product?productId={productId}	DELETE	Delete a product by Id

Table 6: REST endpoints Products

Endpoint	Method	Action
/dev/cart	POST	Create a cart
/dev/cart?userId={userId}	GET	Get a cart by userId
/dev/cart	PUT	Update a cart
/dev/cart?userId={userId}&cartId={cartId}	DELETE	Delete a cart by userId and cartId

Table 7: REST Endpoints for User Cart

Endpoints	Method	Action
API URL/dev/order	POST	Create an order
API URL/dev/order?orderId={orderId}	GET	Get an order by Id
API URL/dev/orders	GET	Get all orders
API URL/dev/order	PUT	Update an order
API URL/dev/order?orderId={orderId}	DELETE	Delete an order by Id

Table 8: REST endpoints Users Orders

9 Summary

The Rair Project is deployed on the AWS cloud platform, which was selected for its scalability, high availability, and cost efficiency. Rair is designed as a single-page application based on Amazon S3 and through the domain https://rair-shop.xyz. The backend follows a Platform as a Service (PaaS) model and is built using serverless frameworks, including AWS API Gateway and Lambda. The agile development was handled using Trello. The development workflow includes DevOps principles, using GitHub for version control and applying the Gitflow strategy to manage branches. Continuous Integration (CI) is simplified within this structure, while deployment to production is handled through pull requests from the test to the main branch, providing consistent code quality checks.

10 Appendix

10.1 Presentation Video

https://drive.google.com/file/d/14jcaH-rbiayGwb4x3TFL5GJHw-YkXx5b/view?usp=drive$_l ink$

10.2 Project Contributors

Contributor	Student ID	GitHub Username
Aymerick Tilly	24244239	AymerickTilly
Isaac Kuku	19085865	010lupin
Ronan Gondipon	23227991	RonanIrvine
Hafizur Rahman	24236822	srj7826

Table 9: First table caption

10.3 Total Expenditure

From mid-April 2025 to early June 2025, the AWS cost breakdown shows a total expenditure of approximately $1.73. An additional $0.50 is expected in July for Route 53 services. This demonstrates that AWS offers cost-effective solutions for running applications.

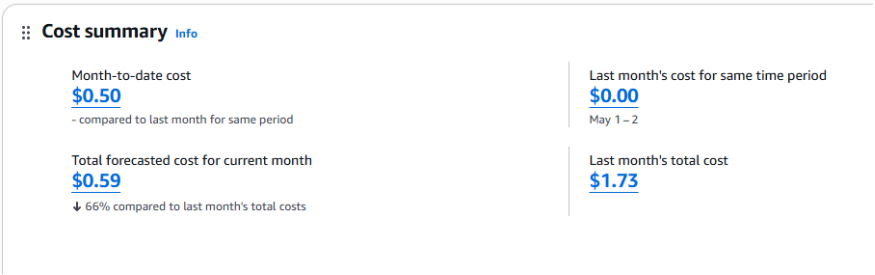

Figure 34: AWS Total Project Expenditure